Trabajo de Taller IISS 2026 — Iteración 4

Arquitectura del controlador (core/engine) y TDD

Alcance general combinado de las iteraciones 4, 5 y 6

Se plantea el alcance general para las próximas 3 iteraciones, orientadas a completar y probar un prototipo del componente para controlar calefacción eléctrica de la aplicación de Domótica IoTEste EcoWarm.

Al finalizar las iteraciones 4 a 6 se espera que cada equipo tenga una versión operativa del prototipo para controlar la temperatura de un conjunto de habitaciones haciendo uso de la información de 1 termostato en cada habitación y de un switch que permite encenderla y apagarla.

El conjunto de habitaciones a controlar y la temperatura objetivo de cada una queda definido por un documento JSON (ver material adjunto) que puede ser obtenido mediante un servicio Rest.

El documento indica datos por habitación como la temperatura deseable y el consumo cuando está encendida, pero también indica datos generales del sitio: el **máximo de consumo eléctrico** soportado por el sitio completo y los **horarios de energía cara** (tarifa de 'punta'), definidos por **día de la semana** (lunes, martes, …).

El componente de control recibe las mediciones de temperatura de cada habitación mediante mensajes MQTT como se prototipó en las entregas anteriores.

El Control de Temperatura actúa sobre los switches mediante servicios REST directos a cada Switch tal como se prototipó en la iteración 3.

El componente de control de temperatura debe ejecutar como un contenedor Docker y demostrar **resiliencia al contexto** para funcionar dentro de los parámetros previstos, **respetando las restricciones** (no superar la carga máxima) y aplicando un **criterio razonable y no azaroso** para decidir qué habitaciones calefaccionar. **No se busca una solución óptima**: el énfasis de las iteraciones 4 a 6 está en los aspectos **no funcionales** —robustez frente a fallas del entorno, reproducibilidad, e integración y validación con software de terceros—, no en la sofisticación del control en sí.

A lo largo de estas iteraciones se documenta la arquitectura con la técnica **4+1** de forma **incremental**: se comienza en esta iteración con la **vista lógica** y un **diagrama de secuencia** que describa el funcionamiento del controlador cuando recibe eventos, y se completan las demás vistas —al menos un diagrama de cada una, mantenido actualizado— hacia las Iteraciones 5 y 6.

Se van a realizar iteraciones basadas en metodología 'Scrumban', con sesiones de planificación al inicio de cada iteración y revisiones semanales posteriores. Cada semana se puede mostrar avance y ajustar alcance con el 'Product Owner'.

Objetivo

Esta iteración busca ampliar las características del controlador y es un requisito la aplicación de TDD (Test Driven Development). Se propone que el controlador tenga al menos dos módulos: - core: lógica que decide qué encender y apagar, sin comunicarse con los switches ni termostatos, utiliza representación interna (modelo) del estado, de los estímulos que llegan y de las decisiones que toma. - engine: módulo que contiene al core y se conecta con el exterior, controla los switches (REST), los interroga (REST), recibe datos de los termostatos (MQTT). También este módulo es responsable de eventos temporales.

En esta iteración se va a recibir una **versión inicial del simulador de integración**, provisto por la cátedra. Este simulador cumple el rol del entorno del controlador: publica los eventos de temperatura por MQTT, expone los switches como servicios REST y ofrece la configuración del sitio por REST (y, más adelante, el tiempo acelerado). A efectos de las pruebas de integración reemplaza al **generador de eventos de termostatos** (Iteración 2) y al **stub del switch** (Iteración 3). Es necesario realizar los ajustes para integrarse correctamente con él. Los servicios y el contrato detallado del simulador se suministrarán junto con esa versión.

Al comenzar la integración con el simulador, el **API REST diseñado en la Iteración 3** (CRUD y comandos) se **estandariza**: se adopta el contrato acordado con el simulador de prueba en lugar del diseño libre de cada equipo.

Características del Controlador — core

Las características centrales del control de temperatura que deben soportarse son:

- Decidir qué habitación debe recibir energía y cuál no.
- En los **cambios de franja horaria**, cortar el consumo al entrar en un horario de energía cara y poder restituirlo al salir.
- Nunca superar el consumo máximo de energía contratado.
- Aplicar un **criterio no azaroso y justificable** para decidir qué habitación recibe energía cuando la carga no alcanza para todas, acercándose de forma razonable a las temperaturas objetivo. No se pretende optimizar.

La aplicación de TDD está directamente vinculada a este módulo, ya que es a este nivel que se debe construir un conjunto de tests unitarios que demuestren cómo se va a comportar el core del controlador cuando, a partir de cierta situación inicial, tenga que responder a distintos estímulos o eventos.

Se debe indicar el criterio por el que se entiende que el conjunto de tests utilizado es adecuado. Los tests deben ser entregados antes del final de la Iteración.

Funcionalidad del Controlador

Además de recibir los eventos de temperatura de los termostatos y de comandar a los switches para encender y apagar las habitaciones, el controlador debe:

- Soportar una operación REST para iniciar o detener el funcionamiento. Al iniciarlo recibe la fecha/hora para sincronizar su funcionamiento.
- Aceptar una operación REST para actualizar la configuración del sitio (habitaciones, temperatura y consumo de cada una). La configuración recibida se **graba localmente** de forma persistente, de modo que el controlador la conserve entre ejecuciones.
- Prever, en el diseño, que el entorno puede fallar (por ejemplo, el MQTT server cae y luego se recupera). El **endurecimiento y las pruebas** de estos casos se realizan en la Iteración 5.

Entregables de la iteración

- Componente de control estructurado en dos módulos: **core** (lógica de decisión, con modelo interno del estado, los estímulos y las decisiones, sin comunicación externa) y **engine** (integra el core, se conecta con switches y termostatos, y gestiona los eventos temporales).
- **Core** con la lógica de control: decidir qué habitaciones reciben energía, cortar el consumo en los horarios de energía cara, no superar el consumo máximo del sitio y acercar cada habitación a su temperatura objetivo.
- **Pruebas unitarias (TDD) sobre el core**, con el **criterio de adecuación** del conjunto de tests documentado. Se entregan antes del final de la iteración.
- Ejecución de las pruebas unitarias, con sus resultados.
- **API REST** (CRUD y comandos) **estandarizado** para la integración con el simulador.
- Operaciones REST para **iniciar/detener** el funcionamiento (al iniciar recibe la fecha/hora de sincronización) y para **actualizar la configuración del sitio**.
- Componente **integrado con la versión inicial del simulador** provisto.
- **Arquitectura 4+1 (inicio):** la **vista lógica** y un **diagrama de secuencia** del manejo de eventos (las demás vistas se completan en It.5-6).

Mecanismo de entrega

- Tag `version4` en GitHub. Copia `.zip` en Moodle.
- Archivos clave:
  - `/modules/core/...` (módulo core, con sus tests unitarios)
  - `/modules/engine/...` (módulo engine)
  - `/docs/tests/criterio-tests.md` (criterio de adecuación del conjunto de tests)
  - `/docs/arquitectura/4mas1.md` (vista lógica + diagrama de secuencia)
  - `/docker/docker-compose.yml` (control + integración con el simulador)
  - `/scripts/build.sh`

- `/README.md` (actualizado: cómo compilar/correr y cómo integrarse con el simulador)

Material adjunto

Los siguientes modelos de datos se entregan como archivos separados en la carpeta `adjuntos/` y se referencian por su nombre:

- `site_config.json` — Ejemplo de configuración del sitio: habitaciones, consumo por habitación, máximo del sitio y horarios de energía cara.
- `sensor.json` — Ejemplo de mensaje MQTT de un sensor H&T (temperatura/humedad).
- `switch_get_status.json` — Ejemplo de status de un switch (estado, potencia, energía).
- `switch_set_request.json` — Ejemplo de comando Set (encender/apagar) al switch.
- `switch_set_response.json` — Ejemplo de respuesta del switch.